UNIVERSITÉ MUNDIAPOLIS
École d'Ingénieurs
Casablanca

2ème Année CI
Génie Informatique
Filière : Informatique

Principes de la Blockchain

Application des principes de la blockchain en Python

Module : Blockchain — TP 02

Preuve d'enjeu (Proof of Stake)

Réalisé par :
Fellah Youssef

Encadré par :
Prof. Louja Ayoub

Année universitaire : 2026/2027

Contents

1 Introduction **1**
1.1 Contexte . . . 1
1.2 Objectifs . . . 1
1.3 Outils et Architecture . . . 1
1.4 Configuration initiale des validateurs . . . 2

2 Mise en place du protocole et Règles de Consensus **2**
2.1 Adhésion et seuil d'engagement (Règle R1) . . . 2
2.2 Sélection du proposant par roulette (Règle R2) . . . 2
2.3 Production d'un bloc et auditabilité (Règles R3 et R4) . . . 3
2.4 Convergence statistique sur 5 000 blocs (Règle R2) . . . 4
2.5 Dynamique des récompenses et risque de centralisation (Règle R5) . . . 4
2.6 Détection des fraudes et Slashing (Règles R6 et R7) . . . 5

3 Tests approfondis du réseau **6**
3.1 Test 1 : Précision selon la taille de l'échantillon . . . 6
3.2 Test 2 : Arrivée de 3 nouveaux validateurs . . . 6
3.3 Test 3 : Emballement par récompense accrue ($r = 1{,}0\,\%$) . . . 7
3.4 Test 4 : Attaque par tirage biaisé et détection fréquentielle . . . 7

4 Synthèse **7**
4.1 Bilan . . . 7
4.2 Limites de la simulation . . . 8
4.3 Améliorations possibles . . . 8

5 Annexes & Bibliographie **8**
5.1 Annexe A : Dépôt du projet . . . 8
5.2 Annexe B : Glossaire . . . 8
5.3 Bibliographie . . . 9

1 Introduction

1.1 Contexte

Dans les protocoles de consensus décentralisés, la **preuve d'enjeu** (*Proof of Stake* ou PoS) constitue une alternative économe en énergie à la preuve de travail (*Proof of Work*). Contrairement au PoW où la production d'un bloc résulte d'une compétition de calcul brute (minage par force brute du nonce), le protocole PoS sélectionne de manière pseudo-aléatoire le producteur de bloc (appelé **proposant** ou *forger*).

La probabilité p_i pour un validateur i d'être désigné pour forger le prochain bloc est rigoureusement proportionnelle à la **mise** (*stake*) qu'il a engagée et verrouillée en garantie dans le réseau :

$$p_i = \frac{m_i}{\sum_j m_j} = \frac{m_i}{M}$$

où m_i désigne la mise du validateur i et M la masse totale des enjeux engagés par l'ensemble des validateurs actifs.

Ce rapport présente l'implémentation, l'expérimentation et l'analyse de sécurité d'un réseau Proof of Stake complet développé en Python avec une architecture client-serveur HTTP, réalisé dans le cadre du TP 02 du module Blockchain.

1.2 Objectifs

Les travaux menés visent à :

1. implémenter les règles d'adhésion au réseau avec seuil d'enjeu minimal (**Règle R1**) ;
2. concevoir l'algorithme déterministe de sélection pondérée par la roulette des mises (**Règle R2**) ;
3. orchestrer la production de blocs transparents et auditables par n'importe quel nœud (**Règles R3 et R4**) ;
4. évaluer l'impact cumulatif des récompenses de forgeage et quantifier le risque de centralisation (**Règle R5**) ;
5. auditer l'intégrité de la blockchain et identifier les blocs falsifiés de manière autonome (**Règle R6**) ;
6. appliquer les sanctions économiques de confiscation de mise et d'exclusion (*slashing*, **Règle R7**) ;
7. tester la robustesse statistique du protocole face à des tricheurs exploitant des tirages biaisés.

1.3 Outils et Architecture

Le réseau fonctionne selon une architecture distribuée interconnectée par le protocole HTTP :

- **Serveur de consensus** (`Server.py`) : hébergé sur un nœud central sous le framework Flask (port 5000), maintenant l'état global du réseau, la liste des validateurs et le grand livre (*ledger*) des blocs produits ;
- **Nœuds validateurs** (`main.ipynb` et notebooks individuels sous `participants/`) : clients autonomes utilisant la bibliothèque `requests` pour interagir avec l'API REST du serveur et réexécuter localement les règles de contrôle ;

- **Reproductibilité** : initialisation du générateur pseudo-aléatoire avec une graine fixe (`GRAINE = 2`) permettant la comparaison formelle des mesures empiriques avec les valeurs théoriques du protocole.

1.4 Configuration initiale des validateurs

Le réseau démarre avec quatre participants dont les mises initiales vérifient la contrainte $m_i \geq 10$ ETH :

Nœud (NUM)	Validateur	Mise initiale (m_i)	Part théorique (p_i)
1	Idriss	40 ETH	40,0 %
2	Moussa	25 ETH	25,0 %
3	Youssef	20 ETH	20,0 %
4	Ayoub	15 ETH	15,0 %
Total	**Réseau (M)**	**100 ETH**	**100,0 %**

Table 1: État initial du pool de validateurs

2 Mise en place du protocole et Règles de Consensus

2.1 Adhésion et seuil d'engagement (Règle R1)

Principe

L'entrée d'un validateur requiert le séquestre d'une garantie minimale ≥ 10 ETH. Cette garantie sert de collatéral économique : elle permet de sanctionner financièrement tout comportement byzantin (malveillant). Le point de terminaison `POST /rejoindre` vérifie cette contrainte et met à jour la mise totale du réseau.

Résultats

Les quatre participants enregistrent successivement leurs enjeux :

```
200 {'mise_totale': 40, 'validateur': {'blocs': 0, 'exclu': False, 'mise': 40, 'nom': 'Idriss'}}
200 {'mise_totale': 65, 'validateur': {'blocs': 0, 'exclu': False, 'mise': 25, 'nom': 'Moussa'}}
200 {'mise_totale': 85, 'validateur': {'blocs': 0, 'exclu': False, 'mise': 20, 'nom': 'Youssef'}}
200 {'mise_totale': 100, 'validateur': {'blocs': 0, 'exclu': False, 'mise': 15, 'nom': 'Ayoub'}}
```

Une tentative d'adhésion avec une mise de 5 ETH est immédiatement rejetée par le serveur :

```
400 {'erreur': 'mise inferieure a 10'}
```

2.2 Sélection du proposant par roulette (Règle R2)

Principe de la roulette déroulée

Avant la création de chaque bloc, le protocole effectue un tirage aléatoire $t \in [0; M]$. On modélise le capital total comme un segment continu $[0; M]$ découpé en intervalles successifs dont la largeur correspond exactement à la mise de chaque validateur. Le validateur sélectionné est celui dont l'intervalle cumulé contient t.



Figure 1: Mécanisme de sélection par roulette pondérée (projection linéaire)

Algorithme

La fonction `choisir_proposant` est une fonction pure : pour un ensemble ordonné de mises et un tirage donné, elle produit invariablement le même résultat, permettant son exécution locale chez n'importe quel nœud vérificateur.

```
def choisir_proposant(mises, tirage):
    cumul = 0.0
    for nom, mise in mises:
        cumul += mise
        if tirage <= cumul:
            return nom
    return mises[-1][0] if mises else None
```

Validation sur tirages de référence

L'évaluation sur les valeurs tests prévues par le sujet confirme l'exactitude de l'intervalle :

- $t = 52{,}3 \in]40; 65] \implies$ **Moussa**
- $t = 12{,}0 \in [0; 40] \implies$ **Idriss**
- $t = 95{,}5 \in]85; 100] \implies$ **Ayoub**

2.3 Production d'un bloc et auditabilité (Règles R3 et R4)

Structure d'un bloc PoS

Contrairement au bloc PoW qui stocke un nonce miné, le bloc PoS consigne la preuve de sa légitimité en intégrant explicitement :

- `numero` : hauteur du bloc dans la chaîne ;
- `proposant` : identité du validateur autorisé à signer ;
- `tirage` : valeur aléatoire générée par le consensus ;
- `mises` : cliché (*snapshot*) complet des mises actives au moment du tirage.

Vérification autonome

L'appel à `POST /produire_bloc` génère le premier bloc :

```
{
  'numero': 1,
  'proposant': 'Ayoub',
  'tirage': 95.6,
  'mises': [['Idriss', 40], ['Moussa', 25], ['Youssef', 20], ['Ayoub',
     15]]
}
```

Tout nœud recalcule localement `choisir_proposant`(mises, 95,6) = "Ayoub", attestant que le bloc respecte la règle R3.

Distribution sur un échantillon court (15 blocs)

La production de 14 blocs supplémentaires permet d'obtenir les 15 premiers blocs de la chaîne (tableau 2).

Bloc	Proposant	Tirage (t)	Intervalle associé	Statut R4
1	Ayoub	95,60	$]85;100]$	Conforme
2	Ayoub	94,78	$]85;100]$	Conforme
3	Idriss	5,66	$[0;40]$	Conforme
4	Idriss	8,49	$[0;40]$	Conforme
5	Youssef	83,55	$]65;85]$	Conforme
6	Youssef	73,60	$]65;85]$	Conforme
7	Youssef	66,97	$]65;85]$	Conforme
8	Idriss	30,81	$[0;40]$	Conforme
9	Moussa	60,59	$]40;65]$	Conforme
10	Moussa	60,68	$]40;65]$	Conforme
11	Moussa	58,12	$]40;65]$	Conforme
12	Idriss	15,84	$[0;40]$	Conforme
13	Moussa	43,07	$]40;65]$	Conforme
14	Idriss	39,35	$[0;40]$	Conforme
15	Youssef	72,30	$]65;85]$	Conforme

Table 2: Historique des 15 premiers blocs générés (seed = 2)

Sur 15 blocs, la distribution observée est : Idriss (5 blocs, 33,3 %), Moussa (4 blocs, 26,7 %), Youssef (4 blocs, 26,7 %) et Ayoub (2 blocs, 13,3 %). Sur un échantillon aussi restreint, la variance statistique est forte : Moussa et Youssef obtiennent le même nombre de blocs alors que leurs mises diffèrent de 5 ETH.

2.4 Convergence statistique sur 5 000 blocs (Règle R2)

Pour valider expérimentalement la convergence de la loi des grands nombres ($p_i \to m_i/M$), 5 000 blocs additionnels sont produits, portant la chaîne à 5 015 blocs sans récompense active ($r = 0$).

Validateur	Mise	Part théorique (p_i)	Part observée	Écart absolu
Idriss	40 ETH	40,0 %	39,5 %	−0,5 %
Moussa	25 ETH	25,0 %	25,2 %	+0,2 %
Youssef	20 ETH	20,0 %	20,7 %	+0,7 %
Ayoub	15 ETH	15,0 %	14,6 %	−0,4 %

Table 3: Comparaison des fréquences de production sur 5 015 blocs

L'écart maximal observé n'excède pas 0,7 point de pourcentage. La proportionnalité du tirage est donc expérimentalement vérifiée.

2.5 Dynamique des récompenses et risque de centralisation (Règle R5)

Formulation mathématique

La règle R5 alloue une récompense au validateur qui forge un bloc : sa mise augmente d'une fraction r :

$$m_i^{(k+1)} = m_i^{(k)} \cdot (1 + r)$$

tandis que les mises des autres validateurs restent stationnaires.

Résultats sur 1 000 blocs ($r = 0{,}3\,\%$)

L'expérience est lancée sur un réseau réinitialisé avec les mêmes 4 validateurs sur 1 000 blocs avec $r = 0{,}003$.

Validateur	Mise dép.	Mise fin	Part dép.	Part fin	Blocs	Facteur (×)
Idriss	40,0 ETH	175,2 ETH	40,0 %	**62,9 %**	493	**4,38**
Moussa	25,0 ETH	50,2 ETH	25,0 %	18,0 %	233	2,01
Youssef	20,0 ETH	31,3 ETH	20,0 %	11,2 %	149	1,56
Ayoub	15,0 ETH	21,8 ETH	15,0 %	7,8 %	125	1,45
Réseau	**100,0 ETH**	**278,5 ETH**	**100,0 %**	**100,0 %**	**1000**	**2,78**

Table 4: Évolution des mises et des parts après 1 000 blocs avec récompenses ($r = 0{,}3\,\%$)

Analyse de l'amplification de centralisation

Le capital global du réseau a cru d'un facteur 2,78. Bien qu'aucun validateur n'ait perdu d'ETH dans l'absolu (Moussa a par exemple doublé son capital, passant de 25 à 50,2 ETH), la part relative de Moussa, Youssef et Ayoub s'est effondrée.

Seul Idriss progresse plus vite que la moyenne du réseau ($\times 4{,}38 > 2{,}78$), portant sa part de 40 % à près de 63 %. C'est l'illustration empirique du biais de centralisation *"rich get richer"* inhérent au Proof of Stake naïf : détenir une mise plus forte confère plus de blocs, donc plus de récompenses, amplifiant l'avantage initial à chaque génération.

2.6 Détection des fraudes et Slashing (Règles R6 et R7)

Attaque par usurpation d'identité (Étape 8)

Un validateur malveillant (Ayoub) force l'inscription d'un bloc à son propre nom via `POST /falsifier`, alors que le tirage attribué ($t = 73{,}60$) se situe dans l'intervalle de Youssef $]65; 85]$). Le bloc falsifié porte le numéro 6.

Algorithme de vérification autonome (Règle R6)

Tout participant audite l'ensemble de la chaîne localement sans solliciter l'avis du serveur :

```
def verifier_bloc(bloc):
    # R6 : recalcule le proposant attendu d'apres le tirage public
    attendu = choisir_proposant(bloc["mises"], bloc["tirage"])
    return bloc["proposant"] == attendu

def verifier_chaine(chaine):
    return [b["numero"] for b in chaine if not verifier_bloc(b)]
```

L'exécution locale produit le verdict détaillé (tableau 5).

Bloc	Signé par	Tirage	Recalcul local R2	Verdict d'audit
1	Ayoub	95,60	Ayoub	ok
2	Ayoub	94,78	Ayoub	ok
3	Idriss	5,66	Idriss	ok
4	Idriss	8,49	Idriss	ok
5	Youssef	83,55	Youssef	ok
6	**Ayoub**	**73,60**	**Youssef**	**FALSIFIE**
7	Youssef	66,97	Youssef	ok
8	Idriss	30,81	Idriss	ok
9	Moussa	60,59	Moussa	ok

Table 5: Audit autonome de la chaîne par recalcul public

Le bloc 6 est immédiatement et formellement identifié comme falsifié :

`recalcul local : [6]  vs  réponse du réseau : [6]`

Exécution de la sanction (Règle R7)

Dès la détection, la sanction est appliquée via `POST /sanctionner` avec fraction = 0,5 :

```
Ayoub : {'exclu': True, 'mise_restante': 7.5, 'perte': 7.5}
mise totale apres exclusion : 85.0 ETH
```

Ayoub subit une pénalité financière immédiate de 7,5 ETH (50 % de sa mise) et est banni de la liste active des validateurs (`exclu = True`). La masse totale d'enjeux retombe à 85 ETH, augmentant automatiquement la part de marché des trois validateurs honnêtes restants.

3 Tests approfondis du réseau

3.1 Test 1 : Précision selon la taille de l'échantillon

La simulation est répétée sur 500 blocs puis sur 100 000 blocs avec les 4 validateurs initiaux :

Validateur	Part théorique	500 blocs	100 000 blocs	Écart (100k)
Idriss	40,0 %	39,4 %	39,9 %	−0,1 %
Moussa	25,0 %	24,6 %	25,0 %	±0,0 %
Youssef	20,0 %	20,0 %	20,1 %	+0,1 %
Ayoub	15,0 %	16,0 %	15,0 %	±0,0 %

Table 6: Convergence de la distribution en fonction du nombre de blocs

L'écart maximal passe de 1,0 point sur 500 blocs à 0,1 point sur 100 000 blocs, démontrant la stricte conformité asymptotique de l'algorithme.

3.2 Test 2 : Arrivée de 3 nouveaux validateurs

Trois validateurs rejoignent le réseau : Aymen (100 ETH), Timan (50 ETH) et Meryem (50 ETH). La mise totale passe à 300 ETH. Sur 5 000 blocs :

- Aymen détient 100/300 = 33,3 % de la mise et forge 33,4 % des blocs ;
- Timan et Meryem détiennent chacun 16,7 % et forgent respectivement 17,3 % et 16,3 % ;
- Les 4 participants d'origine conservent leur mise absolue mais voient leur part relative divisée exactement par 3.

3.3 Test 3 : Emballement par récompense accrue ($r = 1{,}0\,\%$)

En portant le taux de récompense de 0,3 % à 1,0 % sur 1 000 blocs, l'effet d'amplification devient exponentiel :

- Idriss atteint une mise finale de **93 914,3 ETH** (pour 40 ETH au départ), capturant **99,9 %** de l'ensemble de la masse monétaire du réseau ;
- Moussa passe à 69,0 ETH, Youssef à 37,1 ETH et Ayoub à 26,2 ETH : leur capital s'est apprécié mais leur part relative devient nulle (0,0 %).

Ce test démontre qu'un taux de rémunération excessif détruit la décentralisation en quelques centaines de blocs.

3.4 Test 4 : Attaque par tirage biaisé et détection fréquentielle

Scénario d'attaque

Ayoub n'usurpe plus le nom d'autrui sur un tirage public défavorable. À la place, il forge lui-même des tirages artificiels situés systématiquement dans son propre intervalle $[85; 100]$ (1 bloc sur 4).

Incapacité du contrôle R6 unitaire

Chaque bloc étant individuellement cohérent avec le tirage qu'il transporte (`verifier_bloc` = True), la liste des blocs falsifiés reste totalement vide :

```
rep["blocs_falsifies"] = []
```

Détection par analyse statistique

L'attaque est démasquée par l'analyse fréquentielle globale (règle observe − attendu > 3,0 %) :

Validateur	Part de mise	Part des blocs	Écart	Verdict de surveillance
Idriss	40,0 %	28,8 %	−11,2 %	normal
Moussa	25,0 %	18,8 %	−6,2 %	normal
Youssef	20,0 %	16,2 %	−3,8 %	normal
Ayoub	**15,0 %**	**36,2 %**	**+21,2 %**	`SUSPECT`

Table 7: Détection de l'attaque discrète par anomalie de distribution

Alors qu'Ayoub ne dispose que de 15 % de la mise, il génère plus de 36 % des blocs. Les deux modes de contrôle sont strictement complémentaires : la vérification unitaire intercepte les falsifications syntaxiques, tandis que le moniteur de fréquence neutralise la manipulation de l'aléatoire.

4 Synthèse

4.1 Bilan

Ce travail de simulation a permis de concrétiser les concepts fondamentaux du Proof of Stake :

- la **sélection proportionnelle par roulette**, garantissant l'équité mathématique sans gaspillage d'énergie ;

- la **vérification décentralisée R6**, rendue possible par la publicité des tirages et des états de mise enregistrés dans chaque bloc (R4) ;
- l'**effet de centralisation économique R5**, inhérent aux mécanismes de récompense proportionnels ;
- le **slashing R7**, assurant l'alignement des incitations en pénalisant financièrement la trahison.

4.2 Limites de la simulation

Le réseau implémenté présente des simplifications pédagogiques :

- **génération de l'aléatoire** : le tirage est confié au producteur ou au serveur HTTP, ce qui permet la fraude du test 4. En environnement réel, l'aléa doit provenir d'une **fonction aléatoire vérifiable** (VRF) ou d'un schéma de *commit-reveal* (ex. RANDAO sur Ethereum) ;
- **signatures cryptographiques** : les identités reposent sur des chaînes de caractères au lieu de clés publiques et signatures asymétriques (ECDSA / BLS) ;
- **topologie centralisée** : les nœuds communiquent avec un serveur Flask unique au lieu d'un réseau peer-to-peer (P2P) à diffusion par commérage (*gossip protocol*).

4.3 Améliorations possibles

- intégrer une fonction aléatoire vérifiable (VRF) cryptographique garantissant l'impossibilité de truquer le tirage ;
- implémenter un mécanisme de pondération non linéaire (ex. preuve d'enjeu quadratique ou délégation LPoS) pour freiner la centralisation des récompenses ;
- coupler les blocs avec des signatures numériques ECDSA et des arbres de Merkle pour les transactions.

5 Annexes & Bibliographie

5.1 Annexe A : Dépôt du projet

Le code source intégral (notebooks exécutés, implémentation des règles et lanceur serveur) est consultable sur le dépôt GitHub :

`https://github.com/ind4x/BLOCKCHAIN-LAB/tree/main/TP-02`

5.2 Annexe B : Glossaire

Proof of Stake (PoS) Protocole de consensus attribuant la production de blocs selon la proportion de cryptomonnaie mise en gage.

Mise (Stake) Montant d'actifs numériques immobilisé par un validateur pour garantir sa probité sous peine de confiscation.

Proposant (Forger) Validateur tiré au sort pour construire, signer et diffuser le prochain bloc.

Slashing Mécanisme de pénalité confisquant une fraction ou la totalité de la mise d'un nœud en cas de faute avérée.

Roulette des mises Méthode d'échantillonnage découpant un segment continu proportionnellement aux enjeux individuels.

VRF (Verifiable Random Function) Primitive cryptographique générant un nombre aléatoire imprévisible assorti d'une preuve publique de validité.

5.3 Bibliographie

References

[1] S. King et S. Nadal, *PPCoin: Peer-to-Peer Crypto-Currency with Proof-of-Stake*, 2012.

[2] V. Buterin et V. Griffith, *Casper the Friendly Finality Gadget*, arXiv:1710.09437, 2017.

[3] Y. Gilad, R. Hemo, S. Micali, G. Vlachos et N. Zeldovich, *Algorand: Scaling Byzantine Agreements for Cryptocurrencies*, ACM SOSP, 2017.

[4] Pallets Projects, *Flask Documentation (3.1.x) — A lightweight WSGI web application framework*, 2026.

[5] Université Mundiapolis, *TP 02 — Preuve d'enjeu (Proof of Stake)*, Campus Nouaceur, Filière Ingénierie Informatique.